07 — risc8_5stage_core.v Explained: The Top-Level Processor

Oct 6, 2026 · @shiv

1. Overview

`risc8_5stage_core.v` is the whole processor. It creates the six smaller modules, connects them, and holds everything that belongs between stages: the PC, the four pipeline registers, the flags, and the branch, flush, stall and halt logic.

Item	Value
File	design_sources/risc8_5stage_core.v (368 lines)
Module name	risc8_5stage_core (the top level of the design)
Pipeline stages	All five: IF, ID, EX, MEM, WB
Type of logic	Wiring + combinational multiplexers + one clocked block
Called by	tb_risc8_5stage.v, instance dut, lines 46–53; Vivado's design top for synthesis

1.1 Where it sits

```
tb_risc8_5stage.v
└── risc8_5stage_core.v  <== THIS FILE (creates the six below)
    ├── u_imem   : imem          (imem.v)          Stage 1 IF
    ├── u_ctrl   : control_unit  (control_unit.v)  Stage 2 ID
    ├── u_rf     : regfile       (regfile.v)       read in ID, write in WB
    ├── u_hazard : hazard_unit   (hazard_unit.v)   forwarding + stall
    ├── u_alu    : alu           (alu.v)           Stage 3 EX
    └── u_dmem   : dmem          (dmem.v)          Stage 4 MEM
```

1.2 Every module this file calls

Module called	Defined in	Instance name	Lines	Why
imem	imem.v	u_imem	68–71	Fetch the instruction at the PC
control_unit	control_unit.v	u_ctrl	96–112	Decode the opcode into control signals
regfile	regfile.v	u_rf	131–140	Read registers in ID; write the result in WB
hazard_unit	hazard_unit.v	u_hazard	166–192	Choose forwarding sources; detect load-use stalls
alu	alu.v	u_alu	213–220	Compute the result and the Z and C flags
dmem	dmem.v	u_dmem	244–250	Load and store data

No functions or tasks are defined or called in this file; everything else is built from Verilog operators.

1.3 Ports

Port	Dir.	Width	Meaning
clk	in	1	Clock: every register updates on its rising edge
rst	in	1	Synchronous reset, active high: PC = 0, pipeline empty
halted	out	1	Becomes 1 when HLT has reached WB and the pipeline has stopped
pc_out	out	8	Current PC (debug output)
flags_out	out	2	{Z, C} flags (debug output)
instr_done	out	1	1 in each cycle an instruction completes WB (debug output)

1.4 Naming convention used in this file

Prefix	Belongs to	Prefix	Pipeline register
if_	Stage 1 signals	ifid_	IF/ID register
id_	Stage 2 signals	idex_	ID/EX register
ex_	Stage 3 signals	exmem_	EX/MEM register
mem_	Stage 4 signals	memwb_	MEM/WB register
wb_	Stage 5 signals		

2. Walkthrough part 1: ports, Stage 1 (IF) and Stage 2 (ID)

2.1 File header (lines 1–48)

The header lists the five stages, the four pipeline registers, how each hazard is handled, every module this file calls, who calls it (the testbench), and the naming convention from section 1.4.

2.2 Module and ports (lines 50–59)

```
`timescale 1ns/1ps

module risc8_5stage_core (
    input  wire       clk,      // system clock
    input  wire       rst,      // synchronous reset, active high
    output reg        halted,     // 1 after HLT has left the pipeline
    output wire [7:0] pc_out,     // current PC            (debug output)
    output wire [1:0] flags_out,  // {Z, C} status flags   (debug output)
    output wire       instr_done  // 1 when an instruction completes WB
(debug output)
);
```

The processor needs only a clock and a reset; the program is inside imem.v. `halted` is `output reg` because it is set inside the clocked block.

2.3 Stage 1, IF: PC and instruction memory (lines 61–75)

```
    reg  [7:0]  pc;           // program counter
    wire [15:0] if_instr;     // instruction fetched this cycle

    // ---- CALL: module imem from imem.v ----
    imem u_imem (
        .addr  (pc),
        .instr (if_instr)
    );

    // ---- IF/ID pipeline register ----
    reg [15:0] ifid_instr;    // instruction waiting to be decoded
    reg        ifid_valid;    // 0 = bubble
```

- `pc` is an 8-bit register holding the address of the next instruction.
- **Module call:** `imem u_imem (...)` creates one copy of the module `imem` (defined in **imem.v**) and names it `u_imem`. `.addr(pc)` connects imem's `addr` port to the signal `pc`; `.instr(if_instr)` connects its output to `if_instr`. This "`.port(signal)`" style is called **named port connection**.
- `ifid_instr` and `ifid_valid` are the **IF/ID pipeline register**. They are loaded in the clocked block (part 3).

2.4 Stage 2, ID: splitting the instruction (lines 77–87)

```
    // [15:12] opcode | [11:9] rd | [8:6] rs1 | [5:3] rs2 | [7:0] imm8
    wire [3:0] id_opcode = ifid_instr[15:12];
    wire [2:0] id_rd     = ifid_instr[11:9];
    wire [2:0] id_rs1    = ifid_instr[8:6];
    wire [2:0] id_rs2    = ifid_instr[5:3];
    wire [7:0] id_imm8   = ifid_instr[7:0];
```

These wires simply pick bit ranges out of the instruction. `ifid_instr[15:12]` means bits 15 down to 12. No logic is needed, only wiring. Note that `imm8` overlaps rs1 and rs2; each instruction uses only the fields that matter to it.

2.5 Calling the control unit (lines 89–112)

```
    wire [2:0] id_alu_op;
    wire id_alu_src_imm, id_ra_is_rd, id_rb_is_rd, id_uses_a, id_uses_b,
         id_reg_we, id_mem_to_reg, id_mem_we, id_flag_we,
         id_jump, id_br_z, id_br_nz, id_halt;

    // ---- CALL: module control_unit from control_unit.v ----
    control_unit u_ctrl (
        .opcode      (id_opcode),
        .alu_op      (id_alu_op),
        .alu_src_imm (id_alu_src_imm),
        .ra_is_rd    (id_ra_is_rd),
        .rb_is_rd    (id_rb_is_rd),
        .uses_a      (id_uses_a),      // -> hazard_unit (load-use check)
        .uses_b      (id_uses_b),      // -> hazard_unit (load-use check)
        .reg_we      (id_reg_we),
        .mem_to_reg  (id_mem_to_reg),
        .mem_we      (id_mem_we),
        .flag_we     (id_flag_we),
        .jump        (id_jump),
        .br_z        (id_br_z),
        .br_nz       (id_br_nz),
        .halt        (id_halt)
    );
```

First the wires that will carry the control signals are declared. Then **`control_unit u_ctrl`** creates the decoder from **control_unit.v**, feeding it the opcode and connecting each of its 14 outputs to an `id_` wire. These signals are copied into the ID/EX register in part 3.

2.6 Choosing which registers to read (lines 114–118)

```
    // ADDI reads rd on port A; ST reads rd (store data) on port B.
    wire [2:0] id_ra_addr = id_ra_is_rd ? id_rd : id_rs1;
    wire [2:0] id_rb_addr = id_rb_is_rd ? id_rd : id_rs2;
    wire [7:0] id_ra_data, id_rb_data;
```

Two small multiplexers. Normally port A reads rs1 and port B reads rs2. For ADDI the control unit sets `ra_is_rd`, so port A reads rd. For ST it sets `rb_is_rd`, so port B reads rd, the value to store.

2.7 MEM/WB register and calling the register file (lines 120–140)

```
    reg        memwb_valid;
    reg        memwb_reg_we;
    reg  [2:0] memwb_rd;
    reg  [7:0] memwb_data;
    reg        memwb_halt;
    wire [7:0] wb_data = memwb_data;   // Stage 5 result

    // ---- CALL: module regfile from regfile.v ----
    // Read in ID (Stage 2), written in WB (Stage 5).
    regfile u_rf (
        .clk     (clk),
        .rst     (rst),
        .ra_addr (id_ra_addr), .ra_data (id_ra_data),
        .rb_addr (id_rb_addr), .rb_data (id_rb_data),
        // ===== STAGE 5 : WRITE-BACK (WB) =====
        .we      (memwb_reg_we),
        .wr_addr (memwb_rd),
        .wr_data (wb_data)
    );
```

The MEM/WB register is declared here, early, because the register file's write port needs it. In Verilog a signal must be declared before it is used.

`regfile u_rf` creates the register file from **regfile.v**. Its read ports are used by Stage 2 (ID). Its write port is driven by the MEM/WB register, so this connection **is Stage 5 (write-back)**: there is no separate write-back module.

3. Walkthrough part 2: pipeline registers, hazard unit, EX and MEM

3.1 ID/EX and EX/MEM register declarations (lines 142–157)

```
    // ---- ID/EX pipeline register ----
    reg        idex_valid;
    reg  [2:0] idex_alu_op;
    reg        idex_alu_src_imm, idex_uses_a, idex_uses_b;
    reg        idex_reg_we, idex_mem_to_reg, idex_mem_we, idex_flag_we;
    reg        idex_jump, idex_br_z, idex_br_nz, idex_halt;
    reg  [2:0] idex_rd, idex_ra_addr, idex_rb_addr;
    reg  [7:0] idex_ra_data, idex_rb_data, idex_imm;

    // ---- EX/MEM pipeline register (declared early: used by forwarding)
    reg        exmem_valid;
    reg        exmem_reg_we, exmem_mem_to_reg, exmem_mem_we, exmem_halt;
    reg  [2:0] exmem_rd;
    reg  [7:0] exmem_alu_y;       // ALU result
    reg  [7:0] exmem_addr;        // memory address (R[rs1]) for LD / ST
    reg  [7:0] exmem_store_data;  // data to store (R[rd]) for ST
```

These are just declarations of flip-flops; their values are loaded in the clocked block (part 3).

- **ID/EX** (48 bits) holds everything EX and later stages need: the valid bit, 14 control bits, the destination register `rd`, the two register numbers read, the two register values and the immediate.
- **EX/MEM** (32 bits) holds the control bits still needed, `rd`, the ALU result, the memory address and the store data.

3.2 Calling the hazard unit (lines 159–192)

```
    wire [1:0] fwd_a_sel, fwd_b_sel;
    wire       stall;

    // ---- CALL: module hazard_unit from hazard_unit.v ----
    hazard_unit u_hazard (
        // instruction in ID
        .id_valid       (ifid_valid),
        .id_ra_addr     (id_ra_addr),
        .id_rb_addr     (id_rb_addr),
        .id_uses_a      (id_uses_a),
```

```
        .id_uses_b      (id_uses_b),
        // instruction in EX
        .ex_valid       (idex_valid),
        .ex_mem_to_reg  (idex_mem_to_reg),
        .ex_rd          (idex_rd),
        .ex_ra_addr     (idex_ra_addr),
        .ex_rb_addr     (idex_rb_addr),
        .ex_uses_a      (idex_uses_a),
        .ex_uses_b      (idex_uses_b),
        // instruction in MEM
        .mem_reg_we     (exmem_reg_we),
        .mem_mem_to_reg (exmem_mem_to_reg),
        .mem_rd         (exmem_rd),
        // instruction in WB
        .wb_reg_we      (memwb_reg_we),
        .wb_rd          (memwb_rd),
        // results
        .fwd_a_sel      (fwd_a_sel),
        .fwd_b_sel      (fwd_b_sel),
        .stall          (stall)
    );
```

`hazard_unit u_hazard` creates the hazard unit from **hazard_unit.v**. It receives information about the instruction in each stage: ID (from the `id_` wires), EX (from the ID/EX register), MEM (from EX/MEM) and WB (from MEM/WB). It returns the two forwarding selects and `stall`.

3.3 Stage 3, EX: forwarding multiplexers and calling the ALU (lines 194–220)

```
    //   2'b10 -> EX/MEM ALU result   (instruction 1 ahead)
    //   2'b01 -> MEM/WB result       (instruction 2 ahead)
    //   2'b00 -> value read in ID
    wire [7:0] ex_opA = (fwd_a_sel == 2'b10) ? exmem_alu_y :
                        (fwd_a_sel == 2'b01) ? wb_data     : idex_ra_data;
    wire [7:0] ex_opB = (fwd_b_sel == 2'b10) ? exmem_alu_y :
                        (fwd_b_sel == 2'b01) ? wb_data     : idex_rb_data;

    // ---- ALU operand B: immediate or register ----
    wire [7:0] ex_alu_b = idex_alu_src_imm ? idex_imm : ex_opB;
    wire [7:0] ex_alu_y;
    wire       ex_alu_c, ex_alu_z;
```

```
    // ---- CALL: module alu from alu.v ----
    alu u_alu (
        .a      (ex_opA),
        .b      (ex_alu_b),
        .alu_op (idex_alu_op),
        .y      (ex_alu_y),
        .c      (ex_alu_c),
        .z      (ex_alu_z)
    );
```

- `ex_opA` and `ex_opB` are **3-input multiplexers**, written as nested `?:`. The select comes from the hazard unit: newest value from EX/MEM, older from MEM/WB, otherwise the value read in ID.
- `ex_alu_b` chooses between the register value and the immediate (for LDI and ADDI).
- **`alu u_alu`** creates the ALU from **alu.v**. Its operation comes from `idex_alu_op`, which the control unit produced one stage earlier.

3.4 Flags, branch decision and halt detection (lines 222–235)

```
    reg flag_z, flag_c;

    wire take_branch = idex_valid &&
                       (idex_jump || (idex_br_z && flag_z) || (idex_br_nz &&
!flag_z));
    wire [7:0] branch_target = idex_imm;

    // HLT reached EX: stop fetching and squash younger instructions.
    wire ex_halt = idex_valid && idex_halt;
    reg  draining;                          // 1 = HLT seen, waiting for it
to reach WB
    wire fetch_stop = draining || ex_halt;
```

- `flag_z` and `flag_c` are the status flags. They are written at the end of EX (part 3).
- `take_branch` is 1 for a valid JMP, for BZ when Z = 1, or for BNZ when Z = 0. The flags were set by the previous instruction during its EX cycle, so they are already up to date.
- The jump target is the immediate `idex_imm`.
- `ex_halt` is 1 when HLT is in EX. `draining` remembers that HLT was seen, and `fetch_stop` stops fetching from then on.

3.5 Stage 4, MEM: calling the data memory (lines 237–253)

```
    wire [7:0] mem_rdata;
    wire       mem_we = exmem_mem_we && !halted;   // already gated by
'valid'

    // ---- CALL: module dmem from dmem.v ----
    dmem u_dmem (
        .clk   (clk),
        .we    (mem_we),
        .addr  (exmem_addr),
        .wdata (exmem_store_data),
        .rdata (mem_rdata)
    );

    // Result leaving MEM: loaded data for LD, ALU result otherwise.
    wire [7:0] mem_result = exmem_mem_to_reg ? mem_rdata : exmem_alu_y;
```

`dmem u_dmem` creates the data memory from **dmem.v**. It writes only for a valid ST (`exmem_mem_we` already includes the valid bit) and never after halting. `mem_result` picks the loaded byte for LD, or the ALU result for every other instruction, and this goes into MEM/WB.

4. Walkthrough part 3: the clocked block that moves the pipeline

Everything that changes on the clock edge is in **one** `always @(posedge clk)` block (lines 258–361). It has three branches: reset, normal running, and halted.

4.1 Reset (lines 258–277)

```
    always @(posedge clk) begin
        if (rst) begin
            // ---------------- RESET ----------------
            pc          <= 8'h00;
            ifid_instr  <= 16'h0000;  ifid_valid  <= 1'b0;
            idex_valid  <= 1'b0;
            idex_alu_op <= 3'd0; idex_alu_src_imm <= 1'b0;
            idex_uses_a <= 1'b0; idex_uses_b <= 1'b0;
            idex_reg_we <= 1'b0; idex_mem_to_reg <= 1'b0; idex_mem_we <=
1'b0;
            idex_flag_we <= 1'b0; idex_jump <= 1'b0; idex_br_z <= 1'b0;
```

```
            idex_br_nz <= 1'b0; idex_halt <= 1'b0;
            idex_rd <= 3'd0; idex_ra_addr <= 3'd0; idex_rb_addr <= 3'd0;
            idex_ra_data <= 8'h00; idex_rb_data <= 8'h00; idex_imm <= 8'h00;
            exmem_valid <= 1'b0; exmem_reg_we <= 1'b0; exmem_mem_to_reg <=
1'b0;
            exmem_mem_we <= 1'b0; exmem_halt <= 1'b0; exmem_rd <= 3'd0;
            exmem_alu_y <= 8'h00; exmem_addr <= 8'h00; exmem_store_data <=
8'h00;
            memwb_valid <= 1'b0; memwb_reg_we <= 1'b0; memwb_rd <= 3'd0;
            memwb_data  <= 8'h00; memwb_halt <= 1'b0;
            flag_z <= 1'b0; flag_c <= 1'b0;
            draining <= 1'b0; halted <= 1'b0;
```

On reset every register is cleared: the PC starts at 0, every pipeline register becomes a bubble (valid = 0), and the flags and halt state are cleared. Because reset is checked inside `posedge clk`, it is a **synchronous** reset.

4.2 PC and IF/ID: who wins (lines 279–296)

```
        end else if (!halted) begin
            // Priority: branch flush > halt > load-use stall > normal
            if (take_branch) begin
                pc         <= branch_target;   // redirect fetch
                ifid_instr <= 16'h0000;        // flush wrong-path instr
                ifid_valid <= 1'b0;
            end else if (fetch_stop) begin
                ifid_instr <= 16'h0000;        // HLT: fetch nothing more
                ifid_valid <= 1'b0;            // (PC is frozen)
            end else if (stall) begin
                // load-use stall: keep PC and IF/ID unchanged
            end else begin
                pc         <= pc + 8'd1;       // normal sequential fetch
                ifid_instr <= if_instr;
                ifid_valid <= 1'b1;
            end
```

Situation (checked in this order)	PC becomes	IF/ID becomes
Branch or jump taken in EX	branch target	bubble (wrong-path instruction flushed)
HLT in EX, or already draining	unchanged	bubble (nothing more fetched)

Situation (checked in this order)	PC becomes	IF/ID becomes
Load-use stall	unchanged	unchanged (instruction waits)
Normal	PC + 1	the fetched instruction, valid = 1

The branch is checked first: if a branch is taken, the instruction in ID is on the wrong path anyway, so a stall for it no longer matters.

4.3 ID/EX: bubble or copy (lines 298–323)

```
            if (take_branch || fetch_stop || stall) begin
                // Insert a bubble: the instruction in ID is either on the
                // wrong path (branch / HLT) or must wait (stall).
                idex_valid <= 1'b0;
            end else begin
                idex_valid       <= ifid_valid;
                idex_alu_op      <= id_alu_op;
                /* ... every id_ signal is copied to its idex_ register ...
*/
                idex_ra_data     <= id_ra_data;
                idex_rb_data     <= id_rb_data;
                idex_imm         <= id_imm8;
            end
```

If the instruction in ID must not move forward (flush, halt or stall), a **bubble** goes into ID/EX by clearing only the valid bit. Otherwise every decoded signal, register value and the immediate is copied in (lines 304–322 list all 19 assignments).

4.4 EX/MEM and the flags (lines 325–345)

```
            exmem_valid      <= idex_valid;
            exmem_reg_we     <= idex_valid && idex_reg_we;
            exmem_mem_to_reg <= idex_valid && idex_mem_to_reg;
            exmem_mem_we     <= idex_valid && idex_mem_we;
            exmem_halt       <= ex_halt;
            exmem_rd         <= idex_rd;
            exmem_alu_y      <= ex_alu_y;
            exmem_addr       <= ex_opA;    // address  = R[rs1] (forwarded)
            exmem_store_data <= ex_opB;    // ST data  = R[rd]  (forwarded)

            // Flags are updated at the end of EX.
            if (idex_valid && idex_flag_we) begin
                flag_z <= ex_alu_z;
                flag_c <= ex_alu_c;
            end

            // Remember that HLT has been seen.
            if (ex_halt) draining <= 1'b1;
```

Every write-enable is **ANDed with `idex_valid`**, so a bubble can never write a register or memory. The memory address and store data use the *forwarded* operands, so ST and LD always see the newest register values. Flags change only for valid instructions that are allowed to set them.

4.5 MEM/WB and halting (lines 347–361)

```
            memwb_valid  <= exmem_valid;
            memwb_reg_we <= exmem_reg_we;
            memwb_rd     <= exmem_rd;
            memwb_data   <= mem_result;
            memwb_halt   <= exmem_halt;

            // ======== HALT: HLT has reached WB, pipeline is empty ========
            if (memwb_halt) halted <= 1'b1;

        end else begin
            // ---------------- HALTED: freeze ----------------
            memwb_reg_we <= 1'b0;
        end
    end
```

MEM/WB receives the final result. When HLT reaches WB, every older instruction has finished, so `halted` becomes 1. From then on the `else` branch freezes the processor: nothing moves and nothing more is written.

4.6 Debug outputs (lines 363–368)

```
    assign pc_out     = pc;
    assign flags_out  = {flag_z, flag_c};
    assign instr_done = memwb_valid;

endmodule
```

These connect internal signals to output ports, so the testbench (or LEDs on an FPGA board) can watch the PC, the flags, and each instruction completing. `{flag_z, flag_c}` joins the two flags into a 2-bit bus.

5. Mistakes and viva questions

5.1 Common mistakes

- Using a signal before declaring it. That is why MEM/WB and EX/MEM are declared before the modules that use them.
- Forgetting to AND the write-enables with the valid bit: a bubble or flushed instruction could then write a register or memory.
- Giving the stall a higher priority than the branch flush: the pipeline could freeze on a wrong-path instruction.
- Splitting the pipeline registers across several clocked blocks with conflicting conditions. Keeping one clocked block with a clear priority avoids that.
- Using `=` instead of `<=` in the clocked block: registers would update one after another instead of all together.

5.2 Viva questions

Q1. How many modules does the top level instantiate? Six: imem, control_unit, regfile, hazard_unit, alu and dmem.

Q2. What is `.addr(pc)` in an instance? A named port connection: it connects the module's port `addr` to this file's signal `pc`.

Q3. Where is the write-back stage in the code? In the connection of the MEM/WB register to the register file's write port (`.we` , `.wr_addr` , `.wr_data`) inside the `u_rf` instance.

Q4. What is the priority when several events happen at once? Branch flush first, then halt, then load-use stall, then normal operation.

Q5. How is a bubble created? By writing 0 into the valid bit of a pipeline register. Every write-enable is ANDed with that bit, so the bubble does nothing.

Q6. Why are the memory address and store data taken from `ex_opA` and `ex_opB` ? Those are the forwarded operands, so a load or store uses the newest register values even if they have not been written back yet.

Q7. When does `halted` become 1? When HLT reaches the WB stage. By then all older instructions have completed.